

CHAPTER 6

Synchronization Tools

Operating Systems | Exam Preparation Notes
Institute of Information Technology, Jahangirnagar University

Topics Covered in This Chapter

- 1. Introduction to Process Synchronization
- 2. Race Condition and Critical Section
- 3. Critical Section Problem
- 4. Requirements of Synchronization Mechanisms
- 5. Lock Variable Mechanism
- 6. Peterson's Solution and Analysis
- 7. Comparison, Quick Revision, and Exam Questions

1. Introduction to Process Synchronization

Overview

- When two or more processes cooperate, their execution order must be controlled.
- Without control, they can conflict and produce incorrect results.
- A cooperative process can affect or be affected by other processes in the system.
- The mechanism that preserves correct execution order is called **Process Synchronization**.

Definition — Process Synchronization: The procedure of preserving the correct order of execution of cooperative processes so that conflicts do not arise and shared resources are accessed safely.

Definition — Cooperative Process: A process that can affect, or be affected by, the execution of other processes running concurrently in the system.

Race Condition

Definition — Race Condition: A situation where two or more threads or processes read, write, or make decisions based on shared memory concurrently, leading to unpredictable and incorrect results.

- Race conditions occur when the final outcome depends on the scheduling order of processes.
- The result changes each time the program runs, making debugging very difficult.

Example: Two processes both read a shared counter (value = 5), both increment it, and both write back 6. The correct result should be 7. This incorrect output is caused by a Race Condition.

Critical Section

Definition — Critical Section: The region of a program that accesses shared resources such as memory locations, data structures, files, or I/O devices. Only one process may execute inside the Critical Section at any given time.

- Multiple processes trying to execute their critical sections simultaneously causes a Race Condition.
- The solution to synchronization is to properly manage entry and exit from the Critical Section.

QUICK REVISION

- * Process Synchronization controls execution order of cooperative processes.
- * Cooperative process: affects or is affected by other processes.
- * Race Condition: multiple processes access shared data simultaneously, causing wrong output.
- * Critical Section: code region where shared resources are accessed; only one process allowed at a time.

2. Critical Section Problem

Definition

- The Critical Section Problem is the challenge of designing protocols that prevent Race Conditions.
- The Operating System must control which process is allowed to enter the Critical Section at any time.
- The solution must be both correct and fair for all competing processes.

Structure of a Process

A process is divided into four sections: Entry Section, Critical Section, Exit Section, and Remainder Section.

Section	Description
Entry Section	The process requests permission to enter the Critical Section. Synchronization logic is placed here.
Critical Section	The process accesses shared resources. Only one process is permitted here at any time.
Exit Section	The process signals that it has finished with the Critical Section, releasing access for others.
Remainder Section	The rest of the process code that does not involve shared resources. Non-critical work.

The goal is to correctly implement the Entry and Exit sections such that Mutual Exclusion and all other requirements are satisfied.

QUICK REVISION

- * Critical Section Problem: design a protocol so Race Conditions never occur.
- * Four parts of a process: Entry → Critical Section → Exit → Remainder.
- * Only one process is allowed inside the Critical Section at any time.
- * Entry and Exit sections contain all synchronization logic.

3. Requirements of Synchronization Mechanisms

Any valid synchronization solution must satisfy the following requirements. The first two are mandatory.

Requirement	Priority	Description
Mutual Exclusion	Primary — Mandatory	If one process is executing in the Critical Section, no other process may enter it at the same time.
Progress	Primary — Mandatory	If no process is in the Critical Section, a process that wishes to enter must be allowed to do so without unnecessary delay.
Bounded Waiting	Secondary	There must be a limit on how many times other processes can enter the Critical Section before a waiting process is granted access. No process should wait indefinitely.
Portability	Secondary	The solution should be independent of hardware. A software-based solution is inherently portable across different machines.

Mutual Exclusion — Detailed

- If Process P1 is executing inside the Critical Section, no other process (P2, P3, ...) may enter.
- This is the most fundamental requirement. Without it, Race Conditions cannot be prevented.

Example: If P1 is updating a bank account balance, P2 must wait outside the Critical Section until P1 has finished.

Progress — Detailed

- If no process is currently in the Critical Section, a waiting process must not be blocked unnecessarily.
- A process that has no intention of entering the Critical Section must not prevent others from doing so.

Example: If P1 does not need to use the printer, it should not block P2 from accessing it.

Bounded Waiting — Detailed

- A limit must exist on how many times other processes can enter the Critical Section before a waiting process gets its turn.
- This prevents starvation, where one process waits forever while others repeatedly gain access.

Important: Mutual Exclusion and Progress are mandatory requirements. Every valid synchronization solution must satisfy both. Bounded Waiting and Portability are also important but are secondary.

QUICK REVISION

- * Mutual Exclusion: only one process in Critical Section at a time. (Mandatory)
- * Progress: if Critical Section is free, let a waiting process enter. (Mandatory)
- * Bounded Waiting: no process waits forever — starvation must not occur.
- * Portability: software-based solutions work on all hardware platforms.

4. Lock Variable Mechanism

Property	Details
Type	Software mechanism, implemented in user mode
Applicable for	More than two processes
Waiting Strategy	Busy Waiting (process loops continuously checking the lock)
Complexity	Very simple — uses a single shared variable

Definition — Lock Variable: A shared binary variable that can hold the value 0 or 1. A value of 0 indicates the Critical Section is free. A value of 1 indicates it is occupied.

How the Mechanism Works

1. A process wishing to enter the Critical Section first checks the lock value.
2. If lock = 0 (free), the process sets lock = 1 and enters the Critical Section.
3. If lock = 1 (occupied), the process keeps waiting in a loop until lock becomes 0.
4. When the process exits the Critical Section, it sets lock = 0 to release it.

Pseudocode

```
// Entry Section
while (lock != 0);      // Busy wait until lock is 0 (Critical Section is
                        // free)
lock = 1;               // Acquire the lock (mark Critical Section as
                        // occupied)

// Critical Section
// ... access shared resources ...

// Exit Section
lock = 0;               // Release the lock
```

Code Explanation — Line by Line

5. **while (lock != 0)** --> Loop continuously until the Critical Section is free (lock = 0). This is Busy Waiting.
6. **lock = 1** --> Set lock to 1 to signal that the Critical Section is now occupied.
7. **// Critical Section** --> Access shared resources here. Only one process should be here at a time.
8. **lock = 0** --> Release the lock so other processes can enter.

Assembly-Level View (Operating System Perspective)

The OS executes pseudocode as assembly instructions. Examining the assembly reveals why Lock Variable fails.

```

1. Load Lock, R0      ; Read the current value of lock into register R0
2. CMP R0, #0         ; Compare R0 with 0
3. JNZ 1              ; If lock != 0, jump back to step 1 (keep waiting)
4. Store #1, Lock      ; Set lock = 1 (enter Critical Section)
5. Store #0, Lock      ; Set lock = 0 (exit Critical Section)

```

Why Lock Variable Fails — Mutual Exclusion Not Guaranteed

Problem: A process may read the lock value into a register and then be preempted before it can update the lock. Another process then reads the same old lock value and also enters the Critical Section. Both processes are now inside the Critical Section simultaneously, violating Mutual Exclusion.

Failure Scenario — Step-by-Step

Initial condition: lock = 0. Two processes P1 and P2 are competing for the Critical Section.

Step	Process	Action Taken	Lock Value	Outcome
1	P1	Executes step 1 — reads lock = 0 into register R0. Is then preempted.	0	P1 holds R0 = 0 but has not yet set lock = 1
2	P2	Executes steps 1, 2, 3, 4 — reads lock = 0, enters Critical Section.	1	P2 is now inside the Critical Section
3	P1	Resumes. Uses old R0 = 0. Executes steps 2, 3, 4 — also enters CS.	1	BOTH P1 and P2 are inside the Critical Section

Root Cause: P1 stored the old lock value (0) in register R0 before being preempted. When P1 resumed, it used the stale R0 value rather than re-reading the current lock value. As a result, it entered the Critical Section even though P2 was already inside.

Evaluation Against Requirements

Requirement	Satisfied	Reason
Mutual Exclusion	No	Two processes can enter the Critical Section simultaneously, as demonstrated above.
Progress	Yes	If the Critical Section is free, a process can enter without being blocked unnecessarily.
Bounded Waiting	Not applicable	Since Mutual Exclusion is not satisfied, further analysis is not meaningful.
Portability	Yes	It is a pure software-based solution and works on all hardware.

Conclusion: The Lock Variable mechanism fails to satisfy Mutual Exclusion, which is the most fundamental requirement. Therefore, it cannot be used reliably in any real operating system.

QUICK REVISION

- * Lock Variable uses a shared binary variable: 0 = free, 1 = occupied.
- * It is a busy waiting solution applicable to more than two processes.
- * It fails to provide Mutual Exclusion due to the register preemption problem.
- * A process reads the old lock value, is preempted, and another process enters. On resuming, the first process uses its stale register value and also enters the Critical Section.
- * Assembly steps: Load → CMP → JNZ (loop) → Store 1 → Store 0.

5. Peterson's Solution

Property	Details
Type	Software mechanism, implemented in user mode
Applicable for	Exactly two processes only
Waiting Strategy	Busy Waiting
Variables Used	turn variable and interested[] boolean array
Requirements Met	Mutual Exclusion, Progress, Bounded Waiting, Portability

Variables Used

Definition — turn variable: Stores the ID of the process that should wait. The process that last set the turn variable is made to wait, giving priority to the other process.

Definition — interested[] array: A boolean array where interested[i] = TRUE means that process i wants to enter the Critical Section. It is set TRUE before attempting to enter and FALSE after exiting.

Pseudocode — Entry and Exit Sections

```
void Entry_Section(int process)
{
    1.  int other;
    2.  other = 1 - process;           // Identify the other process
    3.  interested[process] = TRUE;    // Signal intent to enter Critical
Section
    4.  turn = process;                // Yield turn to the other process
    5.  while (interested[other] == TRUE && turn == process);
        // Wait if other process is interested AND this process last set turn
}

// Critical Section

void Exit_Section(int process)
{
    6.  interested[process] = FALSE;  // Signal no longer interested
}
```

Code Explanation — Line by Line

9. **1. int other** --> Declare an integer variable to hold the ID of the other process.
10. **2. other = 1 - process** --> If process = 0, then other = 1. If process = 1, then other = 0. Finds the competing process.
11. **3. interested[process] = TRUE** --> Declare that this process wants to enter the Critical Section.
12. **4. turn = process** --> Set the turn variable to the current process. This is a polite gesture — the process that sets turn last will be asked to wait.

13. **5. while (...)** --> Wait (busy wait) only if the other process is also interested AND this process was the last to set turn. If either condition is false, this process may proceed.
14. **6. interested[process] = FALSE** --> After leaving the Critical Section, signal that this process no longer needs access.

Step-by-Step Execution Example

Initial state: interested[0] = FALSE, interested[1] = FALSE, turn = 0.

Step	Process	Action	State After Action
1	P0	Executes lines 1-5. other=1, interested[0]=TRUE, turn=0. Condition in line 5 is FALSE (interested[1] is FALSE). P0 enters Critical Section.	P0 in CS; interested[0]=TRUE; turn=0
2	P0	P0 is preempted while inside Critical Section.	P0 paused in CS
3	P1	Executes lines 1-4. other=0, interested[1]=TRUE, turn=1.	interested[1]=TRUE; turn=1
4	P1	Checks line 5. interested[0]=TRUE AND turn=1. Condition is TRUE. P1 enters busy wait.	P1 waiting; P0 still in CS
5	P0	Resumes. Finishes CS. Executes line 6. Sets interested[0]=FALSE.	P0 exited CS; interested[0]=FALSE
6	P1	Rechecks line 5. interested[0]=FALSE. Condition is now FALSE. P1 enters Critical Section.	P1 in CS

Analysis — Proof that Peterson's Solution is Correct

Mutual Exclusion

- The while loop contains two conditions joined by AND.
- Condition 1: interested[other] == TRUE (the other process also wants the Critical Section).
- Condition 2: turn == process (this process was the last one to set the turn variable).
- Both conditions must be simultaneously TRUE for a process to be held waiting.
- Only one process can be the last to set the turn variable at any given time.
- Therefore, at most one process passes the while loop and enters the Critical Section at a time.

Result: Mutual Exclusion is guaranteed. Two processes can never be inside the Critical Section at the same time.

Progress

- If a process is not interested (interested[i] = FALSE), condition 1 of the while loop is FALSE.
- The other process is not blocked and may enter the Critical Section freely.

- No uninterested process can prevent a willing process from entering.

Result: Progress is guaranteed. If the Critical Section is free, a waiting process is always allowed to enter.

Bounded Waiting

- After a process sets `turn = process`, the other process gets priority (because `turn` now equals the waiting process's ID).
- When the current process exits and sets `interested = FALSE`, the waiting process proceeds immediately.
- A process cannot be bypassed more than once. It will enter the Critical Section within one cycle.

Result: Bounded Waiting is guaranteed. No process waits indefinitely. Starvation cannot occur.

Portability

- Peterson's Solution is entirely software-based. It uses no special hardware instructions.
- It runs correctly on any hardware platform.

Result: Portability is guaranteed.

Overall Conclusion: Peterson's Solution correctly satisfies all four requirements: Mutual Exclusion, Progress, Bounded Waiting, and Portability. It is a valid solution to the Critical Section Problem for two processes.

QUICK REVISION

- * Peterson's Solution is a software-based busy waiting solution for exactly two processes.
- * Two variables: `turn` (which process should wait) and `interested[]` (who wants to enter CS).
- * A process sets `interested[itself] = TRUE` and `turn = itself` before attempting entry.
- * The process that last sets the `turn` variable is made to wait — giving the other process priority.
- * Satisfies all four requirements: Mutual Exclusion, Progress, Bounded Waiting, Portability.
- * Exit: set `interested[process] = FALSE` to release access.

6. Comparison: Lock Variable vs Peterson's Solution

Feature	Lock Variable	Peterson's Solution
Implementation Type	Software — user mode	Software — user mode
Number of Processes	Two or more	Exactly two processes only
Variables Used	One lock variable (0 or 1)	turn variable + interested[] array
Mutual Exclusion	Not provided	Guaranteed
Progress	Provided	Guaranteed
Bounded Waiting	Not applicable (fails first)	Guaranteed
Portability	Yes	Yes
Waiting Strategy	Busy Waiting	Busy Waiting
Suitable for Production	No — unsafe	Yes — for two processes

7. Final Quick Revision Sheet

Important Definitions

Term	Definition
Process Synchronization	Mechanism for controlling the execution order of cooperative processes to prevent conflicts over shared resources.
Cooperative Process	A process that can affect or be affected by other processes running in the system.
Race Condition	A situation where two or more processes access shared data concurrently, producing incorrect or unpredictable results.
Critical Section	The region of a program that accesses shared resources. Only one process may execute here at any given time.
Mutual Exclusion	The guarantee that only one process executes in the Critical Section at a time.
Progress	The guarantee that if the Critical Section is free, a waiting process will be allowed to enter without unnecessary delay.
Bounded Waiting	The guarantee that every process waiting for the Critical Section will eventually gain access — no starvation.
Busy Waiting	A synchronization technique in which a process repeatedly checks a condition in a loop while waiting. Also called spin-lock.
Lock Variable	A shared binary variable (0 = free, 1 = occupied) used as the simplest synchronization mechanism. Fails Mutual Exclusion.
turn variable	In Peterson's Solution: stores the ID of the process that must wait.
interested[] array	In Peterson's Solution: boolean array where interested[i] = TRUE means process i wants to enter the Critical Section.
Preemption	The act of the Operating System forcefully stopping a running process and scheduling another one in its place.

Key Points to Remember

- Lock Variable fails because a process reads the old register value before preemption. On resuming, it enters the Critical Section using the stale value, violating Mutual Exclusion.
- Peterson's Solution uses both turn and interested[] together. The while loop requires BOTH conditions to be TRUE simultaneously, which ensures only one process enters the Critical Section.
- Lock Variable can be applied to more than two processes but is fundamentally unsafe due to its failure on Mutual Exclusion.
- Peterson's Solution is designed for exactly two processes only.

- Both solutions employ Busy Waiting. The process loops in a while loop, consuming CPU cycles while waiting.
- Mutual Exclusion and Progress are the two primary, mandatory requirements. Bounded Waiting is the third essential condition.

Lock Variable Assembly Code

```
1. Load  Lock, R0    -> Read current lock value into register R0
2. CMP    R0, #0      -> Compare R0 with 0
3. JNZ    1           -> If lock != 0, jump back to step 1 (busy wait)
4. Store  #1, Lock    -> Set lock = 1 (acquire lock, enter Critical Section)
5. Store  #0, Lock    -> Set lock = 0 (release lock, exit Critical Section)
```

Peterson's Solution Entry Code

```
1. int other;
2. other = 1 - process;
3. interested[process] = TRUE;
4. turn = process;
5. while (interested[other] == TRUE && turn == process); // busy wait
```

Exit:

```
6. interested[process] = FALSE;
```

8. Possible Exam Questions

Viva / Oral Examination Questions

No.	Question	Key Answer Points
1	What is process synchronization?	Controlling the execution order of cooperative processes to prevent conflicts over shared resources and produce correct results.
2	What is a Race Condition?	Occurs when multiple processes access shared data concurrently. The final result depends on execution order, producing incorrect output.
3	What is a Critical Section?	The region of code that accesses shared resources. Only one process may be inside at any given time.
4	What are the three primary requirements of synchronization?	Mutual Exclusion (one process in CS at a time), Progress (if CS is free, let a waiting process in), Bounded Waiting (no starvation).
5	Why does the Lock Variable mechanism fail?	A process reads the lock value into a register and is preempted. Another process enters the CS. On resuming, the first process uses the stale register value and also enters the CS, violating Mutual Exclusion.
6	What two variables does Peterson's Solution use?	turn: indicates which process should wait. interested[]: boolean array showing which process wants to enter the CS.
7	How does Peterson's Solution guarantee Mutual Exclusion?	The while loop has two AND-conditions. Only one process can be the last to set turn. Therefore, only one can enter the CS at a time.
8	What is Busy Waiting?	A technique where a process continuously loops and checks a condition while waiting to enter the Critical Section. It wastes CPU cycles.
9	For how many processes does Peterson's Solution work?	Exactly two processes. It cannot be extended directly to three or more without modification.
10	What is Bounded Waiting?	A guarantee that every process waiting to enter the Critical Section will do so within a finite, bounded number of attempts. Starvation cannot occur.

Short Answer Questions

15. Define Race Condition and give one example.

- A Race Condition occurs when two or more processes read and write shared data concurrently and the final result depends on the order of execution.
- Example: Two processes simultaneously read a counter value of 5, both increment it, and both write back 6. The correct value should be 7. This incorrect result is caused by a Race Condition.

16. State the three requirements of a synchronization mechanism and classify them.

- Mutual Exclusion (Primary — Mandatory): only one process in the Critical Section at a time.
- Progress (Primary — Mandatory): if the CS is free, a waiting process must be allowed to enter.
- Bounded Waiting (Secondary): no process should wait indefinitely. Starvation must not occur.

17. Explain why the Lock Variable mechanism does not satisfy Mutual Exclusion.

- When a process reads `lock = 0` into its register and is then preempted, another process also reads `lock = 0` and enters the Critical Section.
- When the first process resumes, it uses the stale register value (0) and also enters the Critical Section.
- Both processes are now inside the Critical Section simultaneously. Mutual Exclusion is violated.

18. Write the pseudocode for Peterson's Solution entry and exit sections.

```
Entry_Section(int process):
    other = 1 - process;
    interested[process] = TRUE;
    turn = process;
    while (interested[other] == TRUE && turn == process);

Exit_Section(int process):
    interested[process] = FALSE;
```

19. Prove that Peterson's Solution satisfies Mutual Exclusion.

- The while loop requires: (1) `interested[other] == TRUE` and (2) `turn == process`, both simultaneously.
- At most one process can be the last to have set the turn variable.
- Therefore, at most one process will be blocked by the while loop, and at most one process enters the Critical Section. Mutual Exclusion is proved.

20. Compare Lock Variable and Peterson's Solution using a table.

Refer to the Comparison Table in Section 6 for a complete side-by-side comparison.

21. What is Bounded Waiting? How does Peterson's Solution satisfy it?

- Bounded Waiting ensures that a process will enter the Critical Section within a finite number of turns — no process waits forever.
- In Peterson's Solution, when a process sets `turn = itself`, the other process is given priority. When the current process exits and sets `interested = FALSE`, the waiting process immediately proceeds. A process can be bypassed at most once before it gains access.

Examination Tip: Always address Mutual Exclusion first. It is the most critical requirement and the first condition evaluated when analysing any synchronization solution.